

# LVM Language Specification

Luma Virtual Machine (LVM) - Official Reference Manual

---

**Version:** 1.0

**Architecture:** Stack-based virtual machine

**Word Size:** 16-bit

**Address Space:** 32K (32,768 words)

# 1. Memory Architecture

## 1.1 RAM Organization

| Address Range | Segment | Purpose               | Size          |
|---------------|---------|-----------------------|---------------|
| 0             | SP      | Stack Pointer         | 1 word ▾      |
| 1             | LCL     | Local base address    | 1 word ▾      |
| 2             | ARG     | Argument base address | 1 word ▾      |
| 3             | THIS    | This pointer          | 1 word ▾      |
| 4             | THAT    | That pointer          | 1 word ▾      |
| 5-12          | TEMP    | Temporary variables   | 8 words ▾     |
| 13-15         | R13-R15 | General purpose       | 3 words ▾     |
| 16-255        | STATIC  | Static variables      | 240 words ▾   |
| 256-2047      | STACK   | Working stack         | 1792 words ▾  |
| 2048-16383    | HEAP    | Dynamic allocation    | 14336 words ▾ |
| 16384-24575   | SCREEN  | Screen memory map     | 8192 words ▾  |
| 24576         | KBD     | Keyboard input        | 1 word ▾      |

## 1.2 Virtual Memory Segments

| Segment  | Symbol | Access Method     |
|----------|--------|-------------------|
| local    | LCL ▾  | Base + index ▾    |
| argument | ARG ▾  | Base + index ▾    |
| this     | THIS ▾ | Base + index ▾    |
| that     | THAT ▾ | Base + index ▾    |
| constant | - ▾    | Direct value ▾    |
| static   | - ▾    | Fixed (16-255) ▾  |
| pointer  | - ▾    | THIS/THAT (0/1) ▾ |
| temp     | - ▾    | Fixed (5-12) ▾    |

## 2. Command Types

### 2.1 Overview

| Type         | Category           | Operations    |
|--------------|--------------------|---------------|
| Arithmetic   | Stack manipulation | 9 commands ▾  |
| Memory       | Push/Pop           | 2 commands ▾  |
| Branching    | Program flow       | 3 commands ▾  |
| Function     | Subroutine control | 3 commands ▾  |
| <b>Total</b> |                    | 17 commands ▾ |

### 3. Arithmetic/Logic Commands3.1 Binary Operations

**Syntax:** <operation>

**Stack:** ..., x, y → ..., result

| Command | Operation          | Description         |
|---------|--------------------|---------------------|
| add     | $x + y$            | Integer addition    |
| sub     | $x - y$            | Integer subtraction |
| and     | $x \text{ AND } y$ | Bitwise AND         |
| or      | $x \text{ OR } y$  | Bitwise OR          |

### 3.2 Unary Operations

**Syntax:** <operation>

**Stack:** ..., y → ..., result

| Command | Operation       | Description         |
|---------|-----------------|---------------------|
| neg     | $-y$            | Arithmetic negation |
| not     | $\text{NOT } y$ | Bitwise NOT         |

### 3.3 Comparison Operations

**Syntax:** <operation>

**Stack:** ..., x, y → ..., result

**Returns:** -1 (true) or 0 (false)

| Command | Operation | Description  |
|---------|-----------|--------------|
| eq      | $x == y$  | Equality     |
| gt      | $x > y$   | Greater than |
| lt      | $x < y$   | Less than    |

## 4. Memory Access Commands

### 4.1 Push Command

**Syntax:** `push <segment> <index>`

**Effect:** Push `segment[index]` onto stack

**Segments & Examples:**

- `push constant 17` // Push literal value
- `push local 0` // Push `local[0]`
- `push argument 2` // Push `argument[2]`
- `push this 5` // Push `this[5]`
- `push that 3` // Push `that[3]`
- `push temp 4` // Push `temp[4]` (RAM[9])
- `push pointer 0` // Push THIS
- `push pointer 1` // Push THAT
- `push static 8` // Push static variable

### 4.2 Pop Command

**Syntax:** `pop <segment> <index>`

**Effect:** Pop stack top to `segment[index]`

**Segments & Examples:**

- `pop local 0` // Pop to `local[0]`
- `pop argument 1` // Pop to `argument[1]`
- `pop this 2` // Pop to `this[2]`
- `pop that 6` // Pop to `that[6]`
- `pop temp 3` // Pop to `temp[3]` (RAM[8])
- `pop pointer 0` // Pop to THIS
- `pop pointer 1` // Pop to THAT
- `pop static 5` // Pop to static variable

**Note:** `pop constant` is illegal.

## 5. Program Flow Commands

### 5.1 Label Declaration

**Syntax:** `label <symbol>`

**Effect:** Declare a label

**Examples:**

- `label LOOP_START`
- `label END_IF`
- `label CONTINUE`

### 5.2 Unconditional Jump

**Syntax:** `goto <label>`

**Effect:** Jump to label

**Examples:**

- `goto LOOP_START`
- `goto END_PROGRAM`

### 5.3 Conditional Jump

**Syntax:** `if-goto <label>`

**Stack:** ..., cond → ...

**Effect:** Jump to label if cond ≠ 0

**Examples:**

- `if-goto LOOP_START // Jump if true`
- `if-goto END_IF // Jump if condition met`

## 6. Function Commands

### 6.1 Function Declaration

**Syntax:** `function <functionName> <nVars>`

**Effect:** Declare function with `nVars` local variables

**Examples:**

- `function Main.main 0`
- `function Math.multiply 2`
- `function String.appendChar 1`

### 6.2 Function Call

**Syntax:** `call <functionName> <nArgs>`

**Stack:** `..., arg[nArgs-1], ..., arg[0] → ..., returnValue`

**Effect:** Call function with `nArgs` arguments

**Examples:**

- `call Math.multiply 2 // 2 arguments`
- `call Output.printInt 1 // 1 argument`
- `call Keyboard.readInt 0 // 0 arguments`

### 6.3 Return from Function

**Syntax:** `return`

**Stack:** `..., returnValue → ...`

**Effect:** Return from function

**Example:**

- `return // Return to caller`

## 7. Instruction Reference

### 7.1 Complete Instruction Set

| Category          | Command         | Operands | Stack Effect              |
|-------------------|-----------------|----------|---------------------------|
| <b>Arithmetic</b> |                 |          |                           |
|                   | <b>add</b>      | -        | $x, y \rightarrow x+y$    |
|                   | <b>sub</b>      | -        | $x, y \rightarrow x-y$    |
|                   | <b>neg</b>      | -        | $y \rightarrow -y$        |
|                   | <b>eq</b>       | -        | $x, y \rightarrow (x==y)$ |
|                   | <b>gt</b>       | -        | $x, y \rightarrow (x>y)$  |
|                   | <b>lt</b>       | -        | $x, y \rightarrow (x<y)$  |
|                   | <b>and</b>      | -        | $x, y \rightarrow x\&y$   |
|                   | <b>or</b>       | -        | $x, y \rightarrow x$      |
|                   | <b>not</b>      | -        | $y \rightarrow \sim y$    |
| <b>Memory</b>     |                 |          |                           |
|                   | <b>push</b>     | seg idx  | $\rightarrow \text{val}$  |
|                   | <b>pop</b>      | seg idx  | $\text{val} \rightarrow$  |
| <b>Branching</b>  |                 |          |                           |
|                   | <b>label</b>    | sym      | -                         |
|                   | <b>goto</b>     | sym      | -                         |
|                   | <b>if-goto</b>  | sym      | cond $\rightarrow$        |
| <b>Function</b>   |                 |          |                           |
|                   | <b>function</b> | name n   | -                         |
|                   | <b>call</b>     | name n   | args $\rightarrow$ ret    |
|                   | <b>return</b>   | -        | ret $\rightarrow$         |



## 8. Code Examples

### 8.1 Simple Arithmetic

```
// Compute (7 + 8) - 5

push constant 7
push constant 8
add           // Stack: 15
push constant 5
sub           // Stack: 10
```

### 8.2 Conditional Logic

```
// if (x > 10) y = 1 else y = 0

push local 0      // x
push constant 10
gt
if-goto TRUE_BRANCH

    push constant 0
    pop local 1    // y = 0
    goto END_IF

label TRUE_BRANCH
    push constant 1
    pop local 1    // y = 1

label END_IF
```

### 8.3 Loop Example

```
// sum = 0; for(i=0; i<10; i++) sum += i;
```

```
push constant 0
```

```
pop local 0      // sum = 0
```

```
push constant 0
```

```
pop local 1      // i = 0
```

```
label LOOP_START
```

```
  push local 1
```

```
  push constant 10
```

```
  lt
```

```
  not
```

```
  if-goto LOOP_END
```

```
  push local 0
```

```
  push local 1
```

```
  add
```

```
  pop local 0    // sum += i
```

```
  push local 1
```

```
  push constant 1
```

```
  add
```

```
  pop local 1    // i++
```

```
  goto LOOP_START
```

```
label LOOP_END
```

## 8.4 Function Definition and Call

```
// function multiply(a, b) { return a * b; }
```

```
function Math.multiply 1
```

```
  push constant 0
```

```
  pop local 0      // result = 0
```

```
  label MULT_LOOP
```

```
    push argument 1
```

```
    push constant 0
```

```
    eq
```

```
    if-goto MULT_END
```

```
    push local 0
```

```
    push argument 0
```

```
    add
```

```
    pop local 0    // result += a
```

```
    push argument 1
```

```
    push constant 1
```

```
    sub
```

```
    pop argument 1 // b--
```

```
    goto MULT_LOOP
```

```
  label MULT_END
```

```
  push local 0
```

```
  return
```

```
// Calling the function: z = multiply(3, 4)
```

```
push constant 3
```

```
push constant 4
```

```
call Math.multiply 2
```

```
pop local 2      // z = result
```

## 8.5 Array Access

```
// arr[3] = 17

push constant 3
push argument 0      // Base address of array
add
pop pointer 1        // THAT = arr + 3
push constant 17
pop that 0           // arr[3] = 17
```

## 8.6 Object Method Call

```
// point.setX(5)

push this 0          // Push 'this' pointer
push constant 5
call Point.setX 2     // 2 args: object + value
pop temp 0           // Discard return value
```

## -----Appendix A: Syntax Rules

| Rule       | Description                                 |
|------------|---------------------------------------------|
| Whitespace | Commands separated by newlines              |
| Comments   | <code>// comment text</code>                |
| Case       | Commands are lowercase                      |
| Symbols    | Alphanumeric + underscore, no leading digit |
| Numbers    | Decimal integers (0-32767)                  |

## -----Appendix B: Calling Convention

### Before call:

1. Push arguments (rightmost first)
2. Execute `call` command

### During call (Executed by `function` command):

1. Save return address
2. Save caller's LCL, ARG, THIS, THAT
3. Set ARG = SP - 5 - `nArgs`
4. Set LCL = SP
5. Jump to function

### On return (Executed by `return` command):

1. Push return value
2. Execute `return` command
3. Restore caller's state
4. Continue at return address

## -----End of Specification